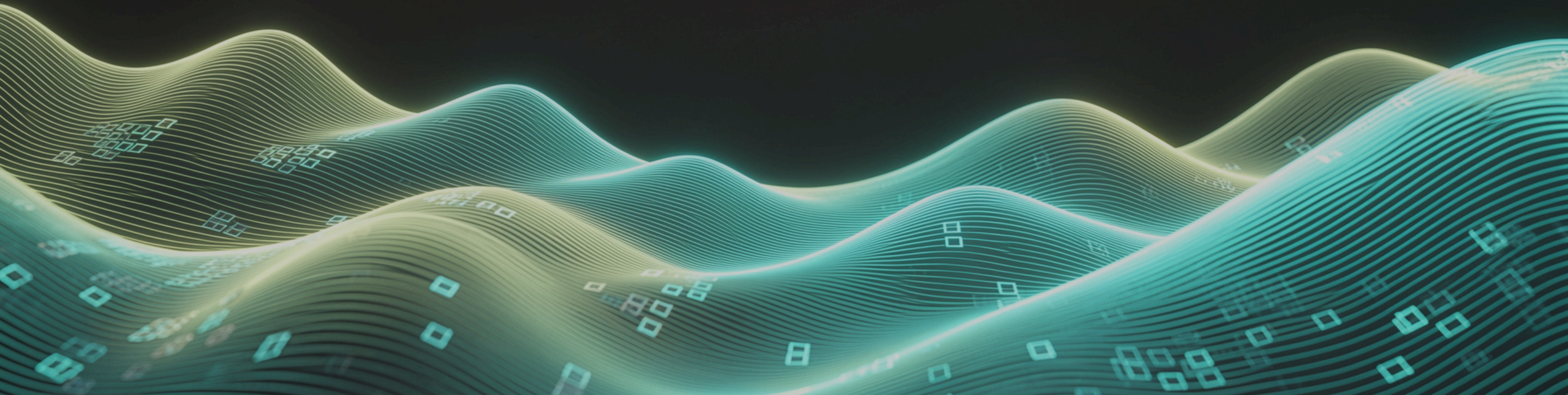


UNIT 1 · MOBILE PROGRAMMING

# State Scope & Logarithmic Recursion

State isolation in C# and binary exponentiation in Dart.

Team 4 · UPEN



# Two problems, one question: where does state live?

---

Two different techniques require the same decision: make what changes during execution explicit.

C#

## Isolate telemetry

Separate execution into reentrant and concurrent calls.

DART

## Reduce recursion

Reduce matrix exponentiation from linear time to  **$O(\log k)$** .

# In C#, global state contaminates execution

A shared static variable mixes metrics across threads and recursive calls.

## 01 MIXING

### Threads + calls

The **Unsafe Engine** can lose counters and confuse depths.

## 02 LEAK

### Shared state

Metrics no longer belong to a specific execution.

## 03 RISK

### Race condition

In critical software, the leak is difficult to reproduce.

| The conceptual reference is the isolation required by deterministic concurrent execution.

# Explicit context makes telemetry reentrant

State travels with the execution; it does not live in a shared static variable.

01

## Create

Create an **ExecutionContext** for a specific execution.

02

## Pass

Pass it directly by reference through the call stack.

03

## Update

Update metrics inside pure functions, without shared static state.

04

## Return

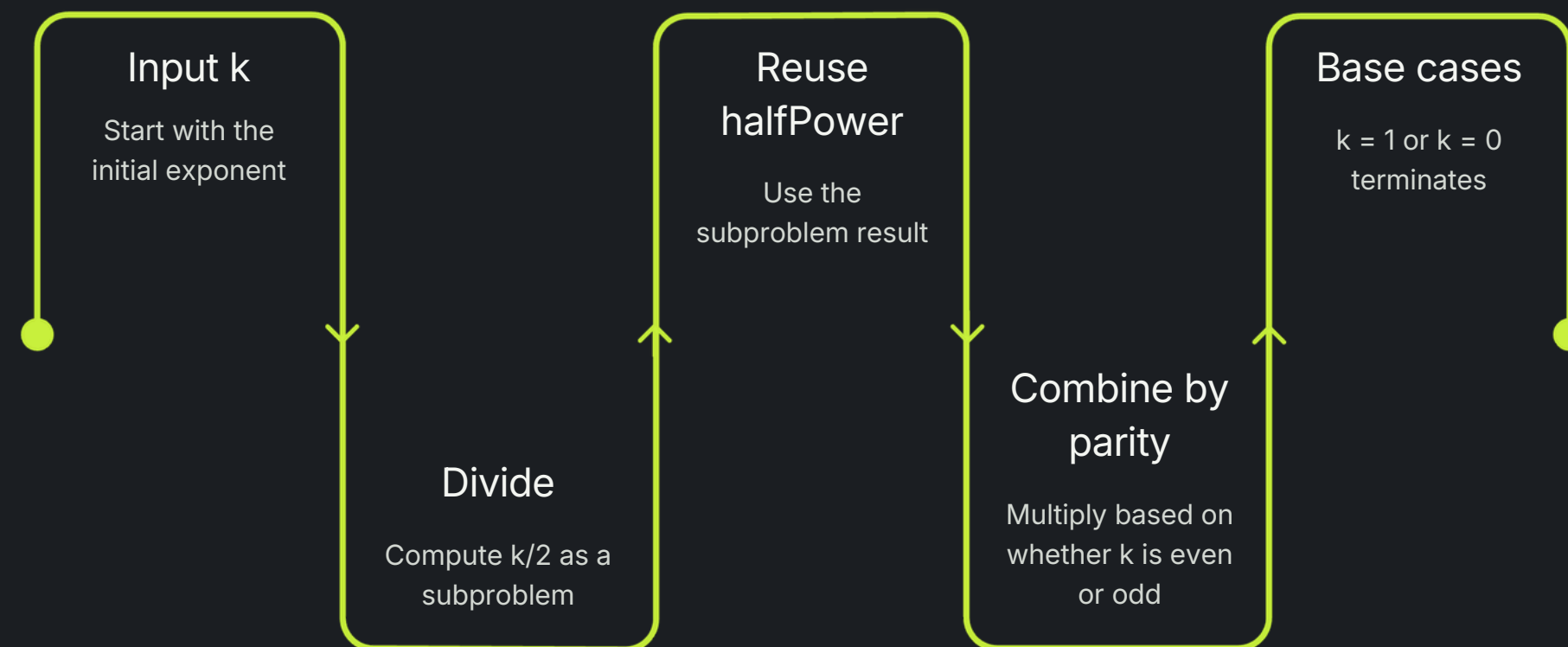
Return the result and the isolated context for each thread.

# The parallel comparison validates the design

Complexity: time **O(N)** and space **O(N)**.

| ENGINE                                 | RECORD                                                   |
|----------------------------------------|----------------------------------------------------------|
| <b>Unsafe Engine</b>                   | Lost counters and mixed recursive depths across threads. |
| <b>Motor Reentrante · Hilo A · 5!</b>  | Recorded a peak of 5 levels.                             |
| <b>Motor Reentrante · Hilo B · 10!</b> | Recorded 10 isolated, deterministic levels.              |

# In Dart, binary exponentiation reduces the call tree



## BASE RULES

**$k = 0$**  returns the identity;  **$k = 1$**  returns  $M$ .

**$k$  even:**  $M^k = M^{(k/2)} \cdot M^{(k/2)}$ .

**$k$  odd:**  $M^k = M \cdot M^{((k-1)/2)} \cdot M^{((k-1)/2)}$ .

Divide and conquer:  **$O(\log k)$** .

# Evaluating `halfPower` once avoids linear recursion

The optimization changes a potentially branching expansion into a logarithmic chain.

01

Calcular

Recursively compute **halfPower** only once.

02

Reutilizar

Reuse the result to build the square.

03

Multiplicar

Multiply by `M` only when the exponent is odd.

04

Registrar

Record depth with **MatrixTelemetry** without duplicating subproblems.

# $F_{60}$ confirms the strategy's power

## FIBONACCI MATRIX

$$F = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

$$F^{60} = \begin{bmatrix} F_{61} & F_{60} \\ F_{60} & F_{59} \end{bmatrix}$$

## RESULT WITH BIGINT

**1,548,008,755,920**

$F_{60}$ , read from  $F^{60}[0][1]$ .

---

**Maximum depth:** 6 levels.

Matches  $\text{floor}(\log_2(60)) + 1 = 6$ .

# Deliverables and conclusion

GitHub repository with **src/CSharp\_Telemetry** and **src/Dart\_MatrixExp**.

**README.md** with team members and an execution guide.

Presentation PDF inside **docs/**.

**Conclusion:** passing context explicitly eliminates side effects in C#, while binary exponentiation in Dart achieves massive logarithmic performance.

